

From Population Models to Personalized Digital Twins

Adapting AI Models to Individuals

Notebook A · build the conditional VAE and then confirm it matches the population-pretrained model.

Notebook A · Dr. Omer Mujahid · UdG–Dexcom Chair for Advancing AI in Clinical Practice

Setup

Run this first. It calls the required libraries.

```
import os
os.environ["TF_USE_LEGACY_KERAS"] = "1" # route tf.keras -> Keras 2 (must precede TF import)

import os
REPO = "diabetes-digital-twins-workshop"
if not os.path.exists(f"/content/{REPO}"):
    !git clone -q https://github.com/omermujahid1/{REPO}.git /content/{REPO}
%cd /content/{REPO}

import numpy as np, pandas as pd, tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras import Model, Input
from tensorflow.keras.initializers import RandomNormal
from tensorflow.keras.layers import (Dense, Dropout, LeakyReLU, Flatten, Lambda,
                                     LSTM, Reshape, Concatenate, TimeDistributed)
from tensorflow.keras import backend as K
from tensorflow.keras.models import load_model

from twin_workshop import (HORIZON, LATENT_DIM, HIDDEN, IN_SHAPE, OUT_SHAPE,
                           MIN_GAIN_MGDL, CONTROL_SCALE, PRETRAINED,
                           list_patients, load_patient, make_windows, holdout_split,
                           verify_setup)

verify_setup()
```

```
/content/diabetes-digital-twins-workshop
Workshop setup check
-----
TensorFlow 2.20.0 | legacy Keras: True
data folder: 1 patients found (csv)
/usr/local/lib/python3.12/dist-packages/tf_keras/src/initializers/initializers.py:121: UserWarning: The initializer RandomNormal is unseeded
warnings.warn(
pretrained decoder loads + forward pass OK
-----
READY
True
```

1 · The encoder $q(z | \text{bg, insulin, carbs})$

Each input is projected to width 18 and stacked into a 3-step sequence `[insulin, carbs, bg]`, read by one recurrent layer into `z_mean` / `z_log_var`.

? Before you run this

Why one sequence, not three encoders?

Insulin, carbs, and BG each get their own Dense projection — but then they're stacked into *one* (3, 18) sequence and read by a single LSTM, rather than encoding each signal separately and concatenating the results. What can a single recurrent layer reading all three stacked together learn that three independent encoders, combined only at the end, could not?

Think about → the LSTM reads the sequence in a fixed order — insulin, then carbs, then BG. Does that order matter, or is it arbitrary?

BLANK 1 Recurrent layer Notebook A · encoder

The stacked `(3, 18)` sequence must be read into a single vector. Use a recurrent layer of width 100 that returns only its final state (not the full sequence).

Your task: call the recurrent layer on `merged`, width 100, final state only.

BLANK 2 Reparameterization trick Notebook A · encoder

You can't backpropagate through a random draw. Rewrite a sample from `N(z_mean, z_var)` as fixed noise scaled by the standard deviation, so gradients still reach `z_mean` and `z_log_var`.

Your task: combine the mean, log-variance, and `eps` into one differentiable expression.

```
def define_encoder():
```

```

def define_encoder():
    init = RandomNormal(stddev=0.02)
    in_bg = Input(shape=OUT_SHAPE, dtype=tf.float32, name="in_bg")
    in_insulin = Input(shape=IN_SHAPE, dtype=tf.float32, name="in_insulin")
    in_carbs = Input(shape=IN_SHAPE, dtype=tf.float32, name="in_carbs")

    n = HORIZON
    bg_r = Reshape((1, n))(Dense(n, kernel_initializer=init)(in_bg))
    ins_r = Reshape((1, n))(Dense(n, kernel_initializer=init)(in_insulin))
    car_r = Reshape((1, n))(Dense(n, kernel_initializer=init)(in_carbs))
    merged = Concatenate(axis=1)([ins_r, car_r, bg_r]) # (3, 18)

    # ---- BLANK 1 ----
    # e = LSTM(units=100, return_sequences=False)(merged) # hint: LSTM, width 100, return_sequences=False
    e = LSTM(100, return_sequences=False, kernel_initializer=init)(merged) # hint: LSTM, width 100, return_sequences=False

    z_mean = Dense(LATENT_DIM, name="z_mean")(e)
    z_log_var = Dense(LATENT_DIM, name="z_log_var")(e)

    def sampling(args):
        zm, zv = args
        eps = tf.random.normal(shape=(tf.shape(zm)[0], tf.shape(zm)[1]))
        # ---- BLANK 2 ----
        return zm + tf.exp(0.5 * zv) * eps # hint: zm + std * eps, std = exp(0.5 * zv)

    z = Lambda(sampling, output_shape=(LATENT_DIM,), name="z")([z_mean, z_log_var])
    return Model([in_bg, in_insulin, in_carbs], [z_mean, z_log_var, z], name="encoder")

encoder = define_encoder(); encoder.summary()

```

Model: "encoder"

Layer (type)	Output Shape	Param #	Connected to
=====			
in_insulin (InputLayer)	[(None, 1, 1)]	0	[]
in_carbs (InputLayer)	[(None, 1, 1)]	0	[]
in_bg (InputLayer)	[(None, 1, 18)]	0	[]
dense_1 (Dense)	(None, 1, 18)	36	['in_insulin[0][0]']
dense_2 (Dense)	(None, 1, 18)	36	['in_carbs[0][0]']
dense (Dense)	(None, 1, 18)	342	['in_bg[0][0]']
reshape_1 (Reshape)	(None, 1, 18)	0	['dense_1[0][0]']
reshape_2 (Reshape)	(None, 1, 18)	0	['dense_2[0][0]']
reshape (Reshape)	(None, 1, 18)	0	['dense[0][0]']
concatenate (Concatenate)	(None, 3, 18)	0	['reshape_1[0][0]', 'reshape_2[0][0]', 'reshape[0][0]']
lstm (LSTM)	(None, 100)	47600	['concatenate[0][0]']
z_mean (Dense)	(None, 5)	505	['lstm[0][0]']
z_log_var (Dense)	(None, 5)	505	['lstm[0][0]']
z (Lambda)	(None, 5)	0	['z_mean[0][0]', 'z_log_var[0][0]']
=====			
Total params: 49024 (191.50 KB)			
Trainable params: 49024 (191.50 KB)			
Non-trainable params: 0 (0.00 Byte)			

2 · The decoder — base trajectory + sign-constrained control

The output is `base(z) + dose-weighted control`. The control weights pass through `softplus` (positive magnitude); the **sign** is then forced so insulin can only lower BG and carbs can only raise it.

BLANK 3 Monotonicity constraint Notebook A · decoder

Both weight streams come out of `softplus(...) + MIN_GAIN_MGDL` as positive magnitudes. Physiology requires the insulin weight to be strictly negative (more insulin lowers BG) while the carb weight stays positive. The carb line is given; write the insulin line.

Your task: force `w_ins` negative by negating the positive magnitude.

? Before you run this

What would an unconstrained model learn?

You're about to force the insulin weight negative by construction, so the model cannot learn that more insulin raises BG, no matter what the training data looks like. Suppose you removed that constraint. In real CGM logs, insulin doses and meals often cluster in time. Could

the model end up learning sign for insulin just by picking up that correlation? What would that mean for anyone later using this model to simulate "what if I gave more insulin"?

Keep this in mind → you'll see exactly this scenario tested in Notebook B's counterfactual section.

```
def define_decoder():
    init = RandomNormal(stddev=0.02)
    seq_len, hidden = HORIZON, HIDDEN
    in_lat = Input(shape=(LATENT_DIM,), dtype=tf.float32, name="in_lat")
    in_ins = Input(shape=IN_SHAPE, dtype=tf.float32, name="in_ins")
    in_carb = Input(shape=IN_SHAPE, dtype=tf.float32, name="in_carb")

    # base branch (trajectory shape from z)
    h = LeakyReLU(0.2)(Dense(hidden, kernel_initializer=init)(in_lat))
    h = LeakyReLU(0.2)(Dense(seq_len * hidden, kernel_initializer=init)(h))
    h = Dropout(0.3)(Reshape((seq_len, hidden))(h))
    base = TimeDistributed(Dense(1, kernel_initializer=init))(h)
    base = Reshape((1, seq_len))(Flatten()(base))

    # control branch (insulin / carb heads)
    c = Concatenate()([Flatten()(in_ins), Flatten()(in_carb)])
    c = LeakyReLU(0.2)(Dense(64, kernel_initializer=init)(c))
    w_raw = Reshape((seq_len, 2))(Dense(seq_len * 2, kernel_initializer=init)(c))
    w_raw = LSTM(16, return_sequences=True)(w_raw)
    w_raw = Dense(2)(w_raw)
    w_ins_raw, w_carb_raw = w_raw[..., 0], w_raw[..., 1]

    w_carb = (K.softplus(w_carb_raw) + MIN_GAIN_MGDL) # given: always positive -> raises BG
    # ---- BLANK 3 ----
    w_ins = -(K.softplus(w_ins_raw) + MIN_GAIN_MGDL) # fixed: use w_ins_raw, not w_carb_raw

    w_ins = Reshape((1, seq_len))(w_ins)
    w_carb = Reshape((1, seq_len))(w_carb)
    ins = Reshape((1, 1))(Flatten()(in_ins))
    carb = Reshape((1, 1))(Flatten()(in_carb))
    control = CONTROL_SCALE * (ins * w_ins + carb * w_carb)

    return Model([in_lat, in_ins, in_carb], base + control, name="decoder_controlled")

decoder = define_decoder(); decoder.summary()
```

Model: "decoder_controlled"

Layer (type)	Output Shape	Param #	Connected to
in_ins (InputLayer)	[(None, 1, 1)]	0	[]
in_carb (InputLayer)	[(None, 1, 1)]	0	[]
flatten_1 (Flatten)	(None, 1)	0	['in_ins[0][0]']
flatten_2 (Flatten)	(None, 1)	0	['in_carb[0][0]']
concatenate_1 (Concatenate)	(None, 2)	0	['flatten_1[0][0]', 'flatten_2[0][0]']
dense_6 (Dense)	(None, 64)	192	['concatenate_1[0][0]']
leaky_re_lu_2 (LeakyReLU)	(None, 64)	0	['dense_6[0][0]']
dense_7 (Dense)	(None, 36)	2340	['leaky_re_lu_2[0][0]']
reshape_5 (Reshape)	(None, 18, 2)	0	['dense_7[0][0]']
in_lat (InputLayer)	[(None, 5)]	0	[]
lstm_1 (LSTM)	(None, 18, 16)	1216	['reshape_5[0][0]']
dense_3 (Dense)	(None, 64)	384	['in_lat[0][0]']
dense_8 (Dense)	(None, 18, 2)	34	['lstm_1[0][0]']
leaky_re_lu (LeakyReLU)	(None, 64)	0	['dense_3[0][0]']
tf.__operators__.getitem_2 (SlicingOpLambda)	(None, 18)	0	['dense_8[0][0]']
dense_4 (Dense)	(None, 1152)	74880	['leaky_re_lu[0][0]']
tf.math.softplus_3 (TFOpLambda)	(None, 18)	0	['tf.__operators__.getitem_2[0][0]']
tf.__operators__.getitem_3 (SlicingOpLambda)	(None, 18)	0	['dense_8[0][0]']
leaky_re_lu_1 (LeakyReLU)	(None, 1152)	0	['dense_4[0][0]']
tf.__operators__.add_5 (TFOpLambda)	(None, 18)	0	['tf.math.softplus_3[0][0]']
tf.math.softplus_2 (TFOpLambda)	(None, 18)	0	['tf.__operators__.getitem_3[0][0]']

reshape_3 (Reshape)	(None, 18, 64)	0	['leaky_re_lu_1[0][0]']
flatten_3 (Flatten)	(None, 1)	0	['in_ins[0][0]']
tf.math.negative_1 (TFOPLa mbda)	(None, 18)	0	['tf.__operators__.add_5[0][0]']

Here's the decoder architecture you just built — the two branches and the sign asymmetry that guarantees monotonicity:

3 · The loss — reconstruction + KL

The trainer adds MSE reconstruction, the KL term, and a soft counterfactual monotonicity penalty. You write the KL divergence between the approximate posterior and the standard normal prior.

BLANK 4 KL divergence Notebook A · loss

For a Gaussian posterior vs. a standard-normal prior, the closed-form KL per latent dim is $-0.5 * (1 + \log_var - \text{mean}^2 - \exp(\log_var))$, summed over the latent dimension and averaged over the batch.

Your task: complete the KL using `z_mean` and `z_log_var`.

```
from tensorflow.keras.optimizers import Adam
from twin_workshop import (KL_WEIGHT, MONO_WEIGHT, LR, INSULIN_UP, CARBS_UP, K_INS_100, K_CARB_100)

class VAETrainer(tf.keras.Model):
    def __init__(self, encoder, decoder, **kw):
        super().__init__()
        self.encoder, self.decoder = encoder, decoder
        self.kl_weight, self.mono_weight = KL_WEIGHT, MONO_WEIGHT
        self.insulin_up, self.carbs_up = INSULIN_UP, CARBS_UP
        self.k_ins_100, self.k_carb_100 = K_INS_100, K_CARB_100

    def train_step(self, data):
        bg, ins, carb = data[0] if isinstance(data, tuple) else data
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder([bg, ins, carb], training=True)
            y0 = self.decoder([z, ins, carb], training=True)
            recon = K.mean(K.square(bg - y0))

            # ---- BLANK 4 ----
            kl = -0.5 * K.mean(K.sum(1 + z_log_var - K.square(z_mean) - K.exp(z_log_var), axis=-1)) # hint: 1 + z_log_var - K.square(z_

            y_ins_up = self.decoder([z, ins * self.insulin_up, carb], training=True)
            y_carb_up = self.decoder([z, ins, carb * self.carbs_up], training=True)
            mu_base, mu_i, mu_c = (K.mean(y0, axis=[1,2]), K.mean(y_ins_up, axis=[1,2]),
                                   K.mean(y_carb_up, axis=[1,2]))
            m_i = (self.insulin_up - 1.0) * self.k_ins_100
            m_c = (self.carbs_up - 1.0) * self.k_carb_100
            mono = K.mean(K.relu(mu_i - (mu_base - m_i)) + K.relu((mu_base + m_c) - mu_c))
            total = recon + self.kl_weight * kl + self.mono_weight * mono
            grads = tape.gradient(total, self.trainable_variables)
            self.optimizer.apply_gradients(zip(grads, self.trainable_variables))
            return {"loss": total, "recon": recon, "kl": kl, "mono": mono}

vae = VAETrainer(encoder, decoder); vae.compile(optimizer=Adam(LR))
print("VAE trainer ready.")
```

VAE trainer ready.

? Look back at what you built

Two safeguards — same job, or different jobs?

The decoder's `w_ins` is architecturally forced negative — that's absolute, no training required. But the loss you just completed *also* adds a `mono` penalty comparing outputs under scaled-up insulin/carbs to baseline. If the sign is already guaranteed, what is `mono` protecting against that the hard constraint doesn't?

Hint → the sign constraint guarantees direction. Look at what `m_i` and `m_c` represent — what property beyond direction are they enforcing?

4 · Confirm against the shipped decoder

Your decoder has the **same architecture** as the population-pretrained one. Load the provided weights into it and run a forward pass — if shapes line up, you've built the right model. (You'll personalize it in Notebook B.)

```
loaded = load_model(PRETRAINED, compile=False)
decoder.set_weights(loaded.get_weights())
z = np.zeros((2, LATENT_DIM), "float32"); s = np.zeros((2, 1, 1), "float32")
print("forward pass output shape:", decoder.predict([z, s, s], verbose=0).shape)
print("Architecture matches the shipped decoder. Continue in Notebook B.")
```

forward pass output shape: (2, 1, 18)
Architecture matches the shipped decoder. Continue in Notebook B.

Empieza a programar o a [crear código](#) con IA.